

עבודה מספר 2 ++C הנחיות הגשה:

1. יש להגיש את כל קבצי העבודה.
2. ההגשה מתבצעת בזוגות. על רק אחד מבין בני הזוג להגיש את העבודה.
בנוסף, נדרש להוסיף בהערות ההגשה בדף ההגשה במודל את השמות והת.ז של שני השותפים.
3. יש להוסיף הוראות ברורות וקריאות למשתמש לפני כל פעולת קלט.
4. יש להקפיד על שמות המחלקות והשיטות כפי שמצוין בעבודה, הקפידו על אותיות גדולות וקטנות.
5. יש להקפיד על תכנות נכון, כולל:
 - שמות משתנים בעלי משמעות.
 - שימוש חוזר בקוד.
 - הקפדה על הזחות נכונות.
 - הוספת הערות באנגלית בלבד. הקפידו לתעד כל שיטה וכל קטע קוד משמעותי.
6. בראש כל קובץ יש להוסיף בהערה את הפרטים הבאים (עם הפרטים שלכם):

Assignment C++: X Author: Israel Israeli, ID: 12345678

*****/

7. כל שאלה יש להפנות לפורום המתאים במודל.
8. הארכות יינתנו רק במקרי מילואים, אבל, או מחלה חמורה ובצירוף אישורים מתאימים.

נושא התרגיל: operator overloading, inheritance, friends

הערה חשובה: רשימת המתודות היא חלקית בלבד.

היא מהווה את המינימום הנדרש. ייתכן ואף סביר להניח שתצטרכו להוסיף מתודות נוספות עבור המחלקות המתוארות. חישבו היטב מה באמת צריך בכל מחלקה.

לכל המחלקות חובה להוסיף Constructor ו- Destructor דיפולטיביים (אפילו אם הוא ריק). על הבנאי הדיפולטיבי לאתחל את האובייקט בערכים הגיוניים. כמו כן עליכם להוסיף מתודות get ו- set **לפי הצורך**.

בתרגיל זה אתם צריכים להשתמש בספריות של vector, string בתרגיל זה אנו נבנה את המחלקות הבאות:

1. מחלקת Product :

צרו מחלקה Product המייצגת מוצר

```
class Product
{
private:
    static int next_id; // Static member to generate unique IDs for products.
    int id;              // Unique ID for each product.
    string name;         // Name of the product.
    double price;        // Price of the product.
    unsigned int quantity; // Quantity available of the product.

public:
    // Constructors:
    Product(const Product &p); // Copy constructor with the same ID.
    Product(const Product &p, int q); // Copy constructor with a limited quantity.
    Product(string name, double price, int q = 1); // Constructor with a new ID.
    double get_price() const; // Returns the price of the product.
    double get_quantity() const; // Returns the quantity of the product.

    double get_id() const; // Returns the quantity of the product.

    // Operator Overloads:
    friend ostream &operator<<(ostream &os, const Product &p); // Prints product details.
    bool operator==(const Product &other) const; // Compares product IDs.
    bool operator==(int other_id) const;
    bool operator!=(const Product &other) const; // Checks inequality of product IDs.
    bool operator!=(int other_id) const;
    Product &operator++(); // Increments the product quantity by 1.
    Product &operator+=(Unsigned int q); // Adds a specific quantity to the product.
    Product &operator--=(unsigned int q); // Adds a specific quantity to the product.

};
```

:ShoppingCart מחלקת 2.

צרו מחלקה ShoppingCart המייצגת עגלת מוצרים

```
// ShoppingCart.h
class ShoppingCart
{
protected:
    vector<Product> items; // List of products in the shopping cart.
    double total_price;    // Total price of products in the cart.

public:
    // Constructors and Destructor:
    ShoppingCart(); // Default constructor initializing an empty cart.
    / virtual ~ShoppingCart(); // Destructor to clean up resources.

    // Methods:
    bool add_Product(const Product &p); // Adds a product to the cart. And update the
total price by (p.price * p.quantity)
    bool add_Product(const Product &p, int quantity); // Adds a product with a specific
quantity. And update the total price by (p.price * quantity)
    bool remove_Product(const Product &p); // Removes a product from the cart.
    bool remove_Product(const Product &p, int quantity); // Removes a specific quantity of
a product. update the total price
    const vector<Product> &Get_List() const; // Returns the list of products in the
cart.

    void printcart() const; // Prints the shopping cart and total price.
    virtual double Get_total() const; // Returns the total price.

    // Operator Overloads:
    friend ostream &operator<<(ostream &os, const ShoppingCart &cart); // Prints cart
details and total price.
    Product* operator[](int id); // Accesses a pointer to product by its ID. If there is
none return nullptr
};
```

: מחלקת Supplier

צרו מחלקה Supplier המייצגת ספק מולו מתבצעות הרכישות.

```
// Supplier.h
class Supplier
{
private:
    double counter=0; // Total money made by customer purchases.
    vector<Product> inventory;
public:
    // Constructors:
    Supplier();// ctor supplier.
    ~Supplier();// dtor supplier.

    // Methods:
    bool remove_Product(const Product &p); // Removes a product from the cart and will
subtract it's price to the counter.
    bool remove_Product(const Product &p, int quantity); // Removes a specific quantity of
a product will subtract it's price to the counter.
    bool customer_purchases(Customer &c); // Updates stock and profit for a single product
add it price to the counter.
    bool customer_purchases(const ShoppingCart &cart);// Updates stock and profit for
multiple products add it total_price to the counter.
    bool change_price(int id, double new_price); // Changes the price of a product by
ID.
    double get_total_profit() const;// Returns the total profit accumulated from customer
purchases.
    // Operator Overloads:
    friend ostream &operator<<(ostream &os, const Supplier &supplier); // Prints supplier
details and total profit.
};
```

4. מחלקת לקוח –

Customer: צרו מחלקת בסיס המייצגת לקוח המחזיק עגלת קניות
BusinessCustomer: ומחלקה יורשת המייצגת לקוח עסקי

לקוח עסקי שונה מלקוח רגיל בכך שהוא כולל מאפיינים נוספים של שם חברה ואחוז הנחה, ודורס את מתודת סיכום הרכישה כדי לחשב את המחיר הסופי לאחר ניכוי ההנחה העסקית.

```
class Customer {
protected:
    string name;           // Customer's name
    ShoppingCart cart;     // Customer's shopping cart (Composition)

public:
    // Constructors and Destructor:
    Customer(string name);
    virtual ~Customer();

    // Methods:
    virtual bool add_to_cart(const Product &p, int quantity); // Adds a product to the
cart
    virtual bool remove_from_cart(int id, int quantity);       // Removes a product from
the cart by ID
    virtual double checkout();                                 // Calculates the total to
pay, empties the cart, and returns the total amount
    void print_cart() const;                                   // Prints the customer's
cart contents

    // Operator Overloads:
    friend ostream &operator<<(ostream &os, const Customer &c); // Prints the customer's
details and cart contents
};

class BusinessCustomer : public Customer {
private:
    string company_name;   // Company name
    double discount_rate;  // Discount rate (e.g., 0.1 for a 10% discount)

public:
    // Constructors and Destructor:
    BusinessCustomer(string name, string company_name, double discount_rate);
    virtual ~BusinessCustomer();

    // Methods:
    virtual double checkout() override; // Override: Calculates the total to pay including
the business discount, empties the cart, and returns the amount

    // Operator Overloads:
    friend ostream &operator<<(ostream &os, const BusinessCustomer &bc); // Prints the
customer's details including company name and discount rate
};
```

5. מחלקת תפריט - Menu

מחלקה זו תנהל את המערכת עליכם ליישם מערכת ניהול תפריט שמאפשרת אינטראקציה בין מוכר לקונה בהתאם. על מחלקה זו לבצע פעולות קלט/פלט מהמשתמש למעט שיטות שהוגדרו לכך במפורש במחלקות אחרות. יש להציג את התפריט בלולאה עד שהמשתמש יבחר לסיים אותה. שיטות:

mainMenu – שיטה זו תדפיס את התפריט הבא:

המשתמש מקיש	שם הפעולה	הערות
1	כניסה לתפריט מוכר.	לאחר הקשה זו המערכת תפעיל מתודה של תפריט מוכר.
2	כניסה לתפריט קונה.	לאחר הקשה זו, המערכת תשאל את המשתמש האם הוא רוצה להיכנס כלקוח רגיל (1) או כלקוח עסקי (2). המערכת תקלוט את פרטי הלקוח (שם, ואם עסקי גם שם חברה ואחוז הנחה), תיצור אובייקט מתאים פולימורפית באמצעות מצביע ל Customer- ותפעיל את תפריט הקונה על אובייקט זה.
3	יציאה מהתוכנית.	לאחר הקשה זו המערכת תדפיס Goodbye! ותצא

SupplierMenu – שיטה זו תדפיס למסך תפריט להפעלת מוכר.

המשתמש מקיש	שם הפעולה	הערות
1	צפייה במוצרים הזמינים.	הקשה זו תדפיס את כל המוצרים הקיימים בספק.
2	הוספת מוצר חדש למלאי.	הקשה זו תדפיס רשימה של שמות כל המוצרים, הכמות שלהם ומספר זיהוי שלהם. בעת הקשה מספר זיהוי מרשימה תינתן האפשרות להוסיף לאותו מוצר כמות. עבור מספר זיהוי שאינו נמצא ברשימה המשתמש יתבקש למלא פרטים עבור מוצר חדש.
3	עדכון מחיר של מוצר קיים.	הקשה זו תדפיס רשימה של שמות כל המוצרים, הכמות שלהם, המחירים שלהם ומספר זיהוי שלהם. בעת הקשה מספר זיהוי מרשימה תינתן האפשרות לעדכן את המחיר. עבור מספר זיהוי שאינו נמצא ברשימה תוחזק הודעת שגיאה.
4	מחיקת מוצר מהמלאי.	הקשה זו תדפיס רשימה של שמות כל המוצרים, הכמות שלהם, המחירים שלהם ומספר זיהוי שלהם. בעת הקשה מספר זיהוי מרשימה תמחק את אותו מוצר מהרשימה. עבור מספר זיהוי שאינו נמצא ברשימה תוחזר הודעת שגיאה.
5	צפייה ברווחים.	הקשה זו תדפיס עבור הספק את הרווח שהוא עשה.
6	חזרה לתפריט הראשי.	יציאה מתפריט זה וחזרה לתפריט הראשי.

buyerMenu – שיטה זו תדפיס למסך תפריט להפעלת הקונה.

המשתמש מקיש	שם הפעולה	הערות
1	צפייה במוצרים הזמינים.	הקשה זו תדפיס את כל מוצרים הקיימים בספק.
2	הוספת מוצר לעגלה.	הקשה זו תדפיס רשימה של כל המוצרים כולל שמות המוצרים הכמות שלהם, המחירים שלהם ומספר הזיהוי שלהם. בעת הקשה מספר זיהוי מרשימה תינתן האפשרות לבחירת כמות של אותו מוצר תתוסף לאותו קונה. עבור מספר זיהוי שאינו נמצא ברשימה תוחזר הודעת שגיאה הפעולה תתבצע באמצעות קריאה למתודות המתאימות של אובייקט ה-Customer.
3	מחיקת מוצר מהעגלה.	הקשה זו תדפיס רשימת המוצרים בעגלה כולל שמות המוצרים הכמות שלהם, המחירים שלהם ומספר זיהוי שלהם. בעת הקשה מספר זיהוי מרשימה תמחק את אותו מוצר מהרשימה. עבור מספר זיהוי שאינו נמצא ברשימה תוחזר הודעת שגיאה. הפעולה תתבצע באמצעות קריאה למתודות המתאימות של אובייקט ה-Customer.
4	צפייה בתוכן העגלה וסך המחיר.	הקשה זו תדפיס רשימת המוצרים בעגלה כולל שמות המוצרים הכמות שלהם, המחירים שלהם ומספר זיהוי שלהם. הפעולה תתבצע באמצעות קריאה למתודות המתאימות של אובייקט ה-Customer.
5	ביצוע קנייה.	הקשה זאת תדפיס מחיר העגלה ואם הוא רוצה לשלם. אם כן - הtotal_price הולך לקופה של המוכר והעגלה מתרוקנת ואם לא הוא חוזר לתפריט הקונה והעגלה נשאר ללא שינוי. הקשה זאת תפעיל את מתודת ה-checkout() הפולימורפית של הלקוח. המתודה תחזיר את הסכום הסופי לתשלום (לאחר חישוב הנחות אם מדובר בלקוח עסקי). סכום זה יתווסף לקופה של הספק, ועגלת הלקוח תתרוקן.
6	יציאה לתפריט הראשי.	יציאה מתפריט זה וחזרה לתפריט הראשי

ה main -

מייצרת אובייקט מסוג תפריט ומריצה את המתודה "תפריט ראשי" וזהו.

הערות:

- הקלטים יהיו מהטיפוסים החוקיים. ז"א בכל מקום נכניס את הטיפוס המתאים. אנחנו לא מתחייבים שהוא יהיה בטווח מסוים – אלא אם כן נאמר אחרת בשאלה עצמה.
 - אחרי כל הדפסה יש לבצע ירידת שורה. יש להקפיד על תכנות נכון:
- a. כל הערכים שהם קבועים (מבחינה לוגית הם לא אמורים להשתנות), חייבים להיות מוגדרים כ: const, define, או enum בהתאם לצורך.

- b. יש לרשום הערות בשפע, ובאנגלית בלבד (לכל מחלקה מה התפקיד של ה, התוכנית מה היא מבצעת, כל פעולה לא טריוויאלית להסביר, וכל 2~3 שורות קוד – הערה, כל מתודה מה היא עושה).
- c. יש להקפיד על כימוס נכון – כל השדות ומתודות השירות ב- private, הממשק ב-public, חלוקה לקבצים.
- d. יש להקפיד על הזחות, כיתוב נכון וקריא, ושמות משמעותיים.
- e. יש לנסות ולייעל את הקוד והתוכנית ככל שניתן. הקפידו על reuse בקוד. נקודות ירדו על קוד כפול !
- f. לפני בקשת קלט, יש להדפיס למשתמש הוראה (איזה קלט מבוקש)
- g. להזכירכם: הנכם נדרשים לתכנת בשפת ++C אי-לכך, כל שימוש בפונקציות וספריות של שפת C אסורה.
3. עבור הצלחת הבדיקות עליכם לוודא שהדפסות זהות לקובץ ההרצה שקיבלתם. (לא יורדו נקודות על שוני ברווחים ושורות).
4. בהצלחה!